

```

-- BASSARI-ZIK : base de données Supabase
-- (à coller dans SQL Editor puis Run)
-- Règles : 5000 chansons max, suppression
à 1 an, disque d'or à 1000 ventes,
-- artiste 75 %, gains retirables après 30
jours, retrait min 1000 F,
-- suspension à 3 signalements,
billetterie.

create extension if not exists pg_cron;

create table profiles(
  id uuid primary key references auth.users
on delete cascade,
  name text not null,
  role text not null check (role in
('fan','artiste')),
  country text not null default
'TG',          -- TG = TMoney, autre =
Visa
  accepted_policy_at timestampz not null
default now(),
  created_at timestampz default now());

create table songs(
  id uuid primary key default
gen_random_uuid(),
  artist_id uuid not null references
profiles(id),
  title text not null,

```

```

    file_path text not
    null,                                -- chemin
    dans le bucket privé "songs"
    status text not null default
    'pending'                            -- pending (500 F non
    payés) | active | suspended | deleted
    check (status in
    ('pending','active','suspended','deleted'))
    ,
    sales_count int not null default 0,
    is_gold boolean generated always as
    (sales_count >= 1000) stored,
    reports_count int not null default 0,
    created_at timestamptz not null default
    now(),
    expires_at timestamptz not null default
    now() + interval '1 year');

create table
payments(                                --
    ref text primary key,               --
    user_id uuid not null references
    profiles(id),
    purpose text not null check (purpose in
    ('hosting','song','sub1','sub3','ticket')),
    target_id uuid,
    amount int not null,
    status text not null default
    'pending',                            -- pending | paid |
    refund_needed
    created_at timestamptz default now());

```

```
create table unlocks(user_id uuid
references profiles(id), song_id uuid
references songs(id), primary
key(user_id,song_id));
create table subscriptions(user_id uuid
primary key references profiles(id),
ends_at timestamptz not null);

create table events(
  id uuid primary key default
  gen_random_uuid(),
  artist_id uuid not null references
  profiles(id),
  title text not null, place text not null,
  event_date date not null,
  price int not null check (price >= 100),
  capacity int not null check (capacity >=
  1),
  sold int not null default 0);
create table tickets(
  id uuid primary key default
  gen_random_uuid(),
  event_id uuid not null references
  events(id), buyer_id uuid not null
  references profiles(id),
  code text not null unique, created_at
  timestamptz default now());

create table sales(
  id uuid primary key default
  gen_random_uuid(),
  kind text not null check (kind in
  ('song','ticket')),
  artist_id uuid not null references
```

```

profiles(id), buyer_id uuid not null
references profiles(id),
    song_id uuid, event_id uuid,
    amount int not null, artist_share int not
null,    -- 75 %
    available_at timestamptz not
null,    -- vente + 30 jours
    created_at timestamptz default now());

create table payouts(
    id uuid primary key default
gen_random_uuid(),
    artist_id uuid not null references
profiles(id),
    amount int not null, method text not
null, account text not null,    -- TMoney ou
Visa
    status text not null default 'pending'
check (status in
('pending','paid','rejected')),
    created_at timestamptz default now());

create table reports(song_id uuid
references songs(id), reporter_id uuid
references profiles(id), primary
key(song_id,reporter_id));

-- ----- Limite de 5000 chansons -----
-----
create function check_song_limit() returns
trigger language plpgsql as $$
begin
    if (select count(*) from songs where
status in ('pending','active','suspended'))

```

```

>= 5000 then
    raise exception 'Capacité maximale
atteinte (5000 chansons)';
    end if; return new;
end $$;
create trigger trg_song_limit before insert
on songs for each row execute function
check_song_limit();

-- ----- Signalements : suspension à 3
-----
create function on_report() returns trigger
language plpgsql security definer set
search_path=public as $$
begin
    update songs set reports_count =
reports_count + 1,
        status = case when reports_count + 1 >=
3 then 'suspended' else status end
    where id = new.song_id and status =
'active';
    return new;
end $$;
create trigger trg_report after insert on
reports for each row execute function
on_report();

-- ----- Paiement confirmé (appelé
UNIQUEMENT par le webhook, rôle service) --
-----
create function fulfill_payment(p_ref text)
returns void language plpgsql security
definer set search_path=public as $$
declare p payments%rowtype; s

```

```

songs%rowtype; e events%rowtype; st text :=
'paid';
begin
    select * into p from payments where ref =
p_ref and status = 'pending' for update;
    if not found then return; end
if;
    -- déjà traité : pas de
double crédit
    if p.purpose = 'hosting' then
        update songs set status='active' where
id=p.target_id and artist_id=p.user_id and
status='pending';
    elsif p.purpose = 'song' then
        select * into s from songs where
id=p.target_id and status='active';
        if found then
            insert into unlocks values
(p.user_id, s.id) on conflict do nothing;
            if found then
                insert into
sales(kind,artist_id,buyer_id,song_id,amoun
t,artist_share,available_at)

values('song',s.artist_id,p.user_id,s.id,p.
amount,round(p.amount*0.75),now()+interval
'30 days');
                update songs set sales_count =
sales_count + 1 where id = s.id;
            end if;
        else st := 'refund_needed'; end if;
    elsif p.purpose in ('sub1','sub3') then
        insert into subscriptions values
(p.user_id, now() + (case p.purpose when
'sub1' then interval '30 days' else

```

```

interval '90 days' end))
    on conflict (user_id) do update set
ends_at = greatest(subscriptions.ends_at,
now())
    + (case p.purpose when 'sub1' then
interval '30 days' else interval '90 days'
end);
    elsif p.purpose = 'ticket' then
        select * into e from events where
id=p.target_id for update;
        if e.sold < e.capacity then
            update events set sold = sold + 1
where id = e.id;
            insert into
tickets(event_id,buyer_id,code) values
(e.id,p.user_id,'BZ-
'||upper(substr(md5(random()::text),1,6)));
            insert into
sales(kind,artist_id,buyer_id,event_id,amou
nt,artist_share,available_at)

values('ticket',e.artist_id,p.user_id,e.id,
p.amount,round(p.amount*0.75),now()+interva
l '30 days');
            else st := 'refund_needed'; end if;
        end if;
        update payments set status = st where ref
= p_ref;
    end $$;
revoke execute on function
fulfill_payment(text) from public, anon,
authenticated;

-- ----- Abonné : débloquent une

```

```

chanson (compte comme 1 vente, 150 F à
l'artiste) -----
create function
unlock_with_subscription(p_song uuid)
returns void language plpgsql security
definer set search_path=public as $$
declare s songs%rowtype;
begin
    if not exists (select 1 from
subscriptions where user_id=auth.uid() and
ends_at>now()) then
        raise exception 'Abonnement inactif';
    end if;
    select * into s from songs where
id=p_song and status='active';
    if not found then raise exception
'Chanson indisponible'; end if;
    insert into unlocks values (auth.uid(),
p_song) on conflict do nothing;
    if found then
        insert into
sales(kind,artist_id,buyer_id,song_id,amoun
t,artist_share,available_at)

values('song',s.artist_id,auth.uid(),p_song
,200,150,now()+interval '30 days');
        update songs set sales_count =
sales_count + 1 where id = p_song;
    end if;
end $$;

-- ----- Solde et retrait (min 1000 F,
gains vieux de 30 jours) -----
create view artist_balance as

```



```

select artist_id,
       coalesce(sum(artist_share) filter (where
available_at <= now()),0)
       - coalesce((select sum(amount) from
payouts p where p.artist_id=s.artist_id and
p.status<>'rejected'),0) as available,
       coalesce(sum(artist_share) filter (where
available_at > now()),0) as pending
from sales s group by artist_id;
alter view artist_balance set
(security_invoker = true);

create function request_payout(p_method
text, p_account text) returns int language
plpgsql security definer set
search_path=public as $$
declare a int;
begin
    select coalesce(available,0) into a from
artist_balance where artist_id =
auth.uid();
    if coalesce(a,0) < 1000 then raise
exception 'Retrait possible à partir de
1000 F'; end if;
    insert into
payouts(artist_id,amount,method,account)
values (auth.uid(),a,p_method,p_account);
    return a;
end $$;

-- ----- Suppression automatique à 1
an (tous les jours à 3 h) -----
select cron.schedule('bz-expire','0 3 * *
* ',

```

```

    $$update songs set status='deleted' where
    expires_at < now() and status <>
    'deleted'$$);
-- Les fichiers audio des chansons
'deleted' sont à effacer par une fonction
serveur (étape suivante).

-- ----- Sécurité (Row Level Security)
-----
create function is_artist() returns boolean
language sql stable security definer set
search_path=public as
$$ select exists(select 1 from profiles
where id=auth.uid() and role='artiste') $$;

alter table profiles enable row level
security;      alter table songs enable row
level security;
alter table payments enable row level
security;      alter table unlocks enable
row level security;
alter table subscriptions enable row level
security; alter table events enable row
level security;
alter table tickets enable row level
security;      alter table sales enable
row level security;
alter table payouts enable row level
security;      alter table reports enable
row level security;

create policy p_prof_read on profiles for
select to authenticated using (true);
create policy p_prof_ins  on profiles for

```

```
insert to authenticated with check (id =
auth.uid());
create policy p_song_read on songs for
select to authenticated using
(status='active' or artist_id=auth.uid());
create policy p_song_ins on songs for
insert to authenticated with check
(artist_id=auth.uid() and is_artist() and
status='pending');
create policy p_pay_read on payments for
select to authenticated using
(user_id=auth.uid());
create policy p_unl_read on unlocks for
select to authenticated using
(user_id=auth.uid());
create policy p_sub_read on subscriptions
for select to authenticated using
(user_id=auth.uid());
create policy p_ev_read on events for
select to authenticated using (true);
create policy p_ev_ins on events for
insert to authenticated with check
(artist_id=auth.uid() and is_artist() and
sold=0);
create policy p_tk_read on tickets for
select to authenticated using
(buyer_id=auth.uid());
create policy p_sale_read on sales for
select to authenticated using
(artist_id=auth.uid());
create policy p_pout_read on payouts for
select to authenticated using
(artist_id=auth.uid());
create policy p_rep_ins on reports for
```

```
insert to authenticated with check
(reporter_id=auth.uid());

-- Stockage audio privé : l'artiste écrit
dans son dossier, personne ne lit
directement
-- (l'écoute passera par des liens signés,
générés après vérification de
l'achat/abonnement).
insert into storage.buckets(id,name,public)
values ('songs','songs',false) on conflict
do nothing;
create policy s_up on storage.objects for
insert to authenticated
  with check (bucket_id='songs' and
(storage.foldername(name))[1] =
auth.uid()::text);
```